Dot — Complete Technical Documentation

A 40-Page Engineering Reference for the Dot AI Web Agent

> *Written for developers with any background — from zero to shipping.*

Author: Juan Kali · University of Regina, B.Sc. Math & CS
Project: Dot — Personal AI Web Agent (Chrome MV3 Extension)
Repository: https://github.com/Kali2007thecodemaster/dot-browser
Stack: TypeScript · React 18 · LangChain.js · Puppeteer · Chrome MV3 · pnpm Workspaces
Upstream Credit: Built on Nanobrowser (Apache 2.0)

Table of Contents

Part I — Foundations

1. What is Dot?

Dot is a **Chrome browser extension** that lets you control a web browser using plain-language instructions. You type something like:

> *"Go to LinkedIn, find software engineer jobs in Toronto posted this week, and save me a list with the company name, title, and apply link."*

And Dot's internal AI agents — a Planner, a Navigator, and a Validator — collaborate to open tabs, read pages, click buttons, scroll, extract data, and return structured results, all without you lifting a finger.

Why does this matter?

Most AI tools are chat interfaces. You ask a question, you get text back. Dot is different: it **acts**. The AI doesn't describe what to do — it does it, inside your real browser, with your logged-in sessions and cookies.

What makes Dot different from raw LLM APIs?

A raw LLM API (like calling `anthropic.messages.create(...)`) gives you back text. It has no memory of the previous message unless you pass the whole conversation, and it cannot take actions in the real world.

Dot wraps LLM calls inside an **agentic loop** — a cycle where the model:

1. Observes the current state of the browser (URL, DOM elements, screenshot)
2. Decides on an action (click, type, navigate, extract)
3. Executes that action
4. Observes the new state
5. Repeats until the task is complete

This loop, with structured outputs and tool definitions, is what transforms a text predictor into a capable web automation agent.

---

2. Core Concepts Glossary

Before reading further, internalize these definitions. They will appear constantly.

Term	Definition
Agent	An LLM that can take actions. It receives observations and outputs structured decisions.
Tool / Action	A function the agent can call. Example: `go_to_url`, `click_element`, `save_results`.
Tool use / Function calling	An LLM feature where instead of returning plain text, the model returns a structured JSON specifying which tool to call with which arguments.
LangChain.js	A JavaScript library that wraps LLM APIs and provides utilities for building agent pipelines, memory management, and tool orchestration.
CDP (Chrome DevTools Protocol)	A protocol that lets external code control a Chrome tab programmatically — navigate, click, read DOM, take screenshots, etc.
Puppeteer	A Node.js library that wraps CDP into a clean API. Used internally by Dot for browser interaction.
Service Worker	The background process of a Chrome MV3 extension. Has no DOM, runs in the background, persists independently of any tab.
Side Panel	A Chrome UI surface that appears as a panel on the right side of the browser. Dot's chat interface lives here.

Port	A long-lived bidirectional message channel between the service worker and the side panel.
Zod	A TypeScript schema validation library. Used to define action input shapes and validate LLM outputs.
pnpm	A fast, disk-efficient Node.js package manager. Uses workspaces to manage multiple packages in one repository.
Turbo	A monorepo build orchestrator. Knows about dependencies between packages and runs tasks in the right order, in parallel when safe.
MV3	Manifest Version 3 — the current Chrome Extension specification. Replaced background pages with service workers.
Agentic loop	The observe → decide → act → observe cycle an AI agent runs through to complete a task.
Snapshot	In web watches, the stored text content of a monitored URL. Compared to the current fetch to detect changes.
createStorage	A factory function that wraps Chrome's storage API and adds live reactivity and TypeScript types.

---

3. Prerequisites and Environment Setup

3.1 What You Need

Tool	Version	Purpose
Node.js	≥ 22.12.0	JavaScript runtime
pnpm	≥ 9.15.1	Package manager
Git	Any	Version control
Google Chrome	Any recent	Loading the extension
A code editor	VS Code recommended	Writing code

3.2 Why pnpm instead of npm?

`npm` installs the same package multiple times across workspaces, wasting disk space. `pnpm` uses **hard links** — one physical copy on disk, many virtual references. In a monorepo with 10 packages that all depend on React, `pnpm` stores React once; `npm` stores it 10 times.

`pnpm` also enforces strict `package.json` declarations. If you forget to list a dependency, your package cannot import it — even if another package has it. This catches bugs early.

3.3 Installation

```
# 1. Install Node.js from nodejs.org, then verify:
node --version   # should print v22.x.x or higher
```

```
# 2. Install pnpm globally:
npm install -g pnpm

# 3. Clone the repo:
git clone https://github.com/Kali2007thecodemaster/dot-browser.git
cd dot-browser

# 4. Install all workspace dependencies:
pnpm install
# pnpm reads pnpm-workspace.yaml, discovers all packages,
# resolves a single lockfile, and links everything together.

# 5. Build the extension:
pnpm build
# Turbo reads turbo.json, determines build order, and runs
# Vite in each package that needs to produce output.
```

3.4 Loading in Chrome

1. Open `chrome://extensions/` in Chrome
2. Toggle **Developer mode** ON (top-right switch)
3. Click **Load unpacked**
4. Navigate to `dot-browser/dist/` and select it
5. The Dot extension appears in your toolbar

> ***Key insight:*** *Chrome loads the `dist/` directory, not the source. Every time you change source code and run `pnpm build`, you must click the **refresh icon** on the extension card in `chrome://extensions/` for Chrome to pick up the new build. During development, `pnpm dev` watches for changes and rebuilds automatically — but you still need to refresh Chrome.*

3.5 Development Mode

```
pnpm dev
# Starts Vite in watch mode for all packages.
# Rebuilds automatically on file saves.
# Still need to reload the extension in chrome://extensions/
```

3.6 Useful Commands

```
# Type-check a single workspace (fastest):
pnpm -F chrome-extension type-check
pnpm -r --filter "./pages/side-panel" type-check

# Build a single workspace:
pnpm -r --filter "./pages/side-panel" build

# Full production build:
pnpm build
```

```
# Create distribution zip:
pnpm zip
```

---

4. Project Structure and Monorepo Architecture

4.1 What is a Monorepo?

A **monorepo** is a single Git repository that contains multiple distinct packages. Contrast this with a **polyrepo** (one repo per package).

Why use a monorepo for Dot?

Dot has several distinct concerns:

- The Chrome extension background worker (agent logic, browser control)
- The side panel UI (React app)
- The options page UI (another React app)
- Shared storage utilities (used by all three)
- Shared TypeScript types
- Build configuration

These could be separate repos, but then changing a storage type would require updating four repos, publishing new versions, and bumping dependencies everywhere. With a monorepo, one commit can change the storage type and all consumers simultaneously.

4.2 Directory Layout

```
dot-browser/
│
├── chrome-extension/          # Service worker + agent logic
│   ├── manifest.js            # Generates manifest.json at build time
│   ├── src/
│   │   └── background/
│   │       ├── index.ts       # Service worker entry point
│   │       ├── agent/         # The three agents + executor
│   │       │   ├── executor.ts
│   │       │   ├── planner.ts
│   │       │   ├── navigator.ts
│   │       │   ├── validator.ts
│   │       │   ├── types.ts   # AgentContext, shared state
│   │       │   ├── actions/
│   │       │   │   ├── builder.ts   # All action implementations
│   │       │   │   └── schemas.ts   # Zod schemas for every action
│   │       │   └── prompts/
│   │       │       └── templates/   # System prompt strings
│   │       └── browser/
│   │           ├── context.ts       # Multi-tab management
│   │           └── dom/             # DOM tree extraction
│   └── public/                # Static assets (icons)
│
├── pages/
│   ├── side-panel/            # The chat UI (React app)
```

```
│   │   └── src/
│   │       ├── SidePanel.tsx  # Root component / state machine
│   │       ├── SidePanel.css  # Theme + layout
│   │       └── components/    # All UI components
│   ├── options/               # Settings page (React app)
│   └── content/               # Content script (injected into pages)
│
├── packages/
│   ├── storage/               # Chrome storage abstractions + stores
│   │   └── lib/
│   │       ├── base/          # createStorage factory
│   │       ├── profileStore.ts
│   │       ├── resultsStore.ts
│   │       ├── watchStore.ts
│   │       ├── scheduledTaskStore.ts
│   │       └── uploadStore.ts
│   ├── i18n/                  # Internationalization strings
│   ├── ui/                    # Shared React components
│   └── tailwind-config/       # Shared Tailwind configuration
│
├── docs/                      # Documentation (you are here)
├── pnpm-workspace.yaml        # Tells pnpm which directories are packages
├── turbo.json                 # Build pipeline configuration
└── package.json               # Root scripts
```

4.3 pnpm-workspace.yaml

```yaml
# pnpm-workspace.yaml
packages:
  - 'chrome-extension'
  - 'pages/*'
  - 'packages/*'
```

This file tells pnpm: "*Every directory matching these patterns is a package.*" When you run `pnpm install` at the root, pnpm reads all the `package.json` files in these directories and builds a unified dependency graph.

4.4 turbo.json — Build Orchestration

```json
{
  "tasks": {
    "build": {
      "dependsOn": ["^build"],
      "outputs": ["dist/**"]
    },
    "type-check": {
      "dependsOn": ["^build"]
    }
  }
}
```

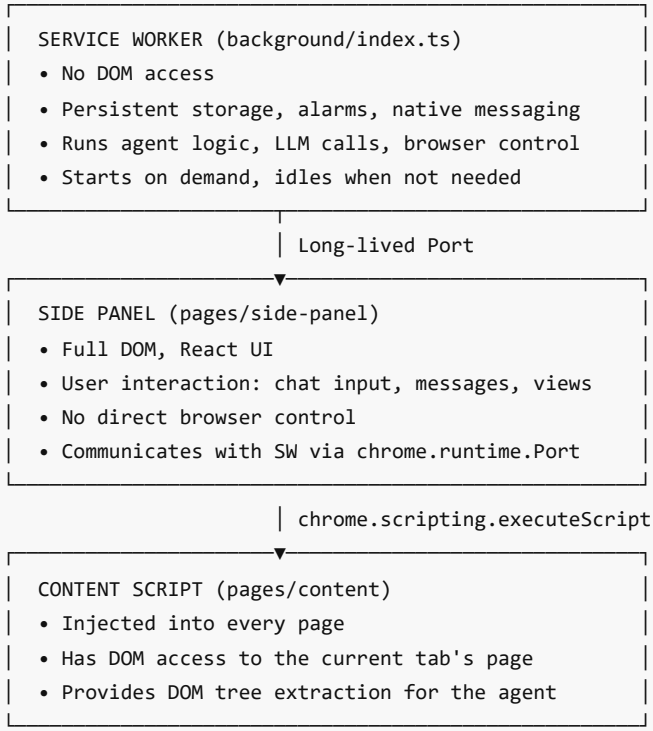
`"dependsOn": ["^build"]` means: "*Before building this package, build all packages it depends on.*" The `^` prefix means "from dependencies." So if `chrome-extension` depends on `@extension/storage`, Turbo builds `storage` first, then `chrome-extension`. It parallelizes independent packages.

---

5. Chrome Extension Architecture (MV3)

5.1 The Three Contexts

A Chrome extension does not run as a single process. It has three distinct execution contexts, each with different capabilities:



5.2 The Manifest

The manifest file is the extension's identity card. Dot generates it dynamically at build time:

```
// chrome-extension/manifest.js (simplified)
const manifest = {
  manifest_version: 3,
  name: "__MSG_app_metadata_name__",  // i18n key
  version: "0.1.0",

  // What the extension is allowed to do:
  host_permissions: ["<all_urls>"],   // Access any website
  permissions: [
    "storage",        // chrome.storage API
    "scripting",      // Execute scripts in tabs
    "tabs",           // Read tab info (URL, title, ID)
    "activeTab",      // Access the current focused tab
```

```
    "debugger",       // CDP - browser control
    "alarms",         // Schedule recurring tasks
    "notifications",  // Show OS notifications
    "contextMenus",   // Right-click menu entries
    "sidePanel",      // The side panel surface
    "webNavigation",  // Listen to navigation events
    "unlimitedStorage"// No storage quota limit
  ],

  // The service worker:
  background: {
    service_worker: "background.iife.js",
    type: "module"
  },

  // The side panel:
  side_panel: {
    default_path: "side-panel/index.html"
  },

  // Scripts injected into web pages:
  content_scripts: [{
    matches: ["http://*/*", "https://*/*"],
    all_frames: true,
    js: ["content/index.iife.js"]
  }]
};
```

> ***Why `__MSG_app_metadata_name__`?*** *Chrome supports i18n in manifests. The `__MSG_key__` syntax is replaced at runtime with the value from `_locales/en/messages.json`. This lets you ship one extension that shows the correct name in different languages.*

5.3 Service Worker Lifecycle

In MV3, background scripts are **service workers** — they don't run continuously. Chrome starts them when an event fires (a message arrives, an alarm triggers, a tab updates) and stops them when idle. This saves memory but creates a challenge: **you cannot keep state in memory between invocations**.

Dot uses `chrome.storage.local` for all persistent state (watches, tasks, sessions). The service worker re-reads storage on every activation rather than relying on in-memory variables.

5.4 Message Passing Architecture

Chrome extensions communicate via message passing. Dot uses two patterns:

One-shot messages (for simple requests):

```
// Side panel or component sends:
chrome.runtime.sendMessage({ type: 'register_watch', watchId: 'abc', intervalMinutes: 60 });

// Service worker receives:
chrome.runtime.onMessage.addListener((message, _sender, sendResponse) => {
  if (message.type === 'register_watch') {
```

```
    chrome.alarms.create(`dot-watch-${message.watchId}`, {
      delayInMinutes: message.intervalMinutes
    });
    sendResponse({ ok: true });
    return true; // <-- important: keeps channel open for async response
  }
});
```

Long-lived ports (for streaming events during a task):

```
// Side panel connects once:
const port = chrome.runtime.connect({ name: 'side-panel-connection' });

// Service worker accepts the connection:
chrome.runtime.onConnect.addListener(port => {
  if (port.name === 'side-panel-connection') {
    port.onMessage.addListener(message => {
      // Handle: new_task, cancel_task, follow_up_task, etc.
    });
  }
});

// During task execution, the service worker streams events:
port.postMessage({ type: 'execution', state: 'STEP_START', actor: 'NAVIGATOR', ... });
```

The long-lived port is essential because tasks can take minutes. The side panel needs a continuous stream of status updates — "planner is thinking", "navigator is clicking", "validator is checking" — not just a final answer.

Part II — The Agent System

6. What is an AI Agent?

6.1 From Chat to Action

A basic LLM interaction looks like this:

```
User: "What is the capital of France?"
Model: "Paris."
```

This is **question-answering** — the model returns text, nothing happens in the world.

An **agent** is a system where the model can:

1. Access tools (functions it can call)
2. Observe the results of those tool calls
3. Decide which tool to call next based on those results
4. Repeat until a goal is reached

```
User: "Book me a flight from Toronto to Paris for next Friday"
```

```
Agent Loop:
  → Observe: "I'm on a blank page. I need to navigate somewhere."
  → Act: go_to_url("https://google.com/flights")
  → Observe: "I see a search form with 'From', 'To', and 'Date' fields."
  → Act: input_text(index=1, text="Toronto")
  → Observe: "Autocomplete showed 'Toronto Pearson International Airport'"
  → Act: click_element(index=5)  // Select the autocomplete suggestion
  ... (continues until task is complete or agent decides it's done)
```

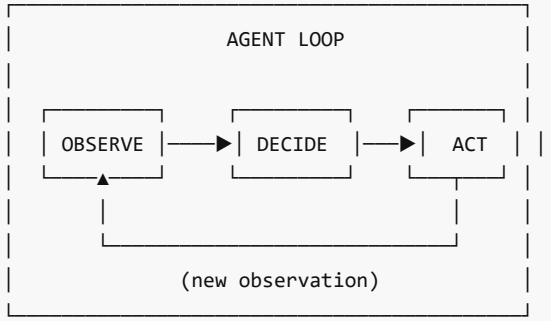
6.2 Tool Use / Function Calling

Modern LLMs like Claude and GPT-4 support **structured outputs** — instead of free text, the model returns a JSON object specifying a function call:

```
{
  "action": [{
    "go_to_url": {
      "intent": "Navigate to Google Flights to search for flights",
      "url": "https://google.com/flights"
    }
  }]
}
```

The application parses this JSON, executes the function (`go_to_url`), captures the result, and feeds it back to the model as context for the next decision. This cycle is **tool use** or **function calling**.

6.3 The Observe-Decide-Act Cycle



Each iteration, the agent:

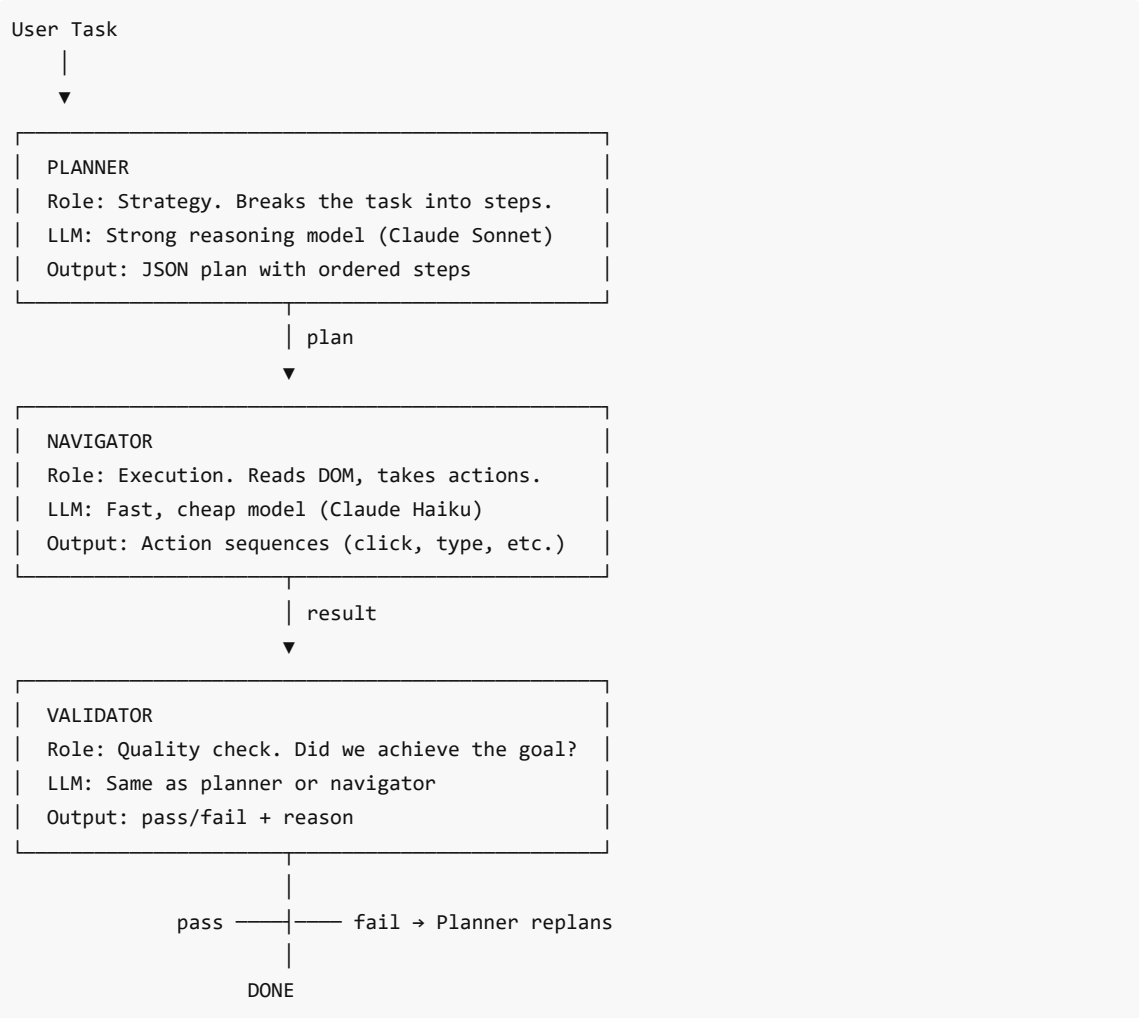
- **Observes**: Current URL, page title, list of interactive elements, screenshot (if vision enabled), previous action history
- **Decides**: Which action to take next, expressed as structured JSON
- **Acts**: The framework executes the action against the real browser

---

7. The Three-Agent System

7.1 Why Three Agents?

A single LLM handling planning, navigation, and validation simultaneously would require a very long context window, mixing high-level strategy ("find all jobs") with low-level execution ("click element index 42") and quality checking ("did we actually get all jobs?"). Separating concerns into specialized agents produces better results.



7.2 The Planner Agent

File: `chrome-extension/src/background/agent/planner.ts`

The Planner receives:

- The user's original task
- The current page URL and title
- Any previous step history
- Memory from previous planning rounds

It outputs a **JSON plan**: an ordered list of steps, each with a description and expected outcome.

```
// Simplified example of a plan the Planner might output:
{
  "plan": {
    "goal": "Find software engineer jobs in Toronto on LinkedIn",
    "next_steps": [
      "Navigate to LinkedIn Jobs search page",
```

```
      "Enter 'software engineer' in the job title field",
      "Enter 'Toronto' in the location field",
      "Filter by 'Past 24 hours' to get recent postings",
      "Extract job titles, companies, and apply links from the results"
    ],
    "completed_steps": []
  }
}
```

System prompt excerpt:

```
// chrome-extension/src/background/agent/prompts/templates/planner.ts
export const plannerSystemPromptTemplate = `
You are a strategic planning agent. Your role is to:
1. Break down complex user tasks into clear, actionable steps
2. Consider the current browser state and history
3. Produce a JSON plan that the Navigator can execute

# FINANCIAL SAFETY
Before planning any step that involves: purchases, payments,
subscriptions, form submission with financial data — you MUST
include a human_interrupt step. No exceptions.
`;
```

7.3 The Navigator Agent

File: `chrome-extension/src/background/agent/navigator.ts`

The Navigator is the workhorse. It receives:

- The current step from the Planner's plan
- The current browser state (URL, DOM element list, screenshot)
- The full list of available actions (tools)

It outputs one or more actions to execute:

```
{
  "current_state": {
    "evaluation_previous_goal": "Success — I navigated to LinkedIn Jobs",
    "memory": "On LinkedIn Jobs search page. Need to fill in the search form.",
    "next_goal": "Type 'software engineer' into the job title search box"
  },
  "action": [
    {
      "input_text": {
        "intent": "Fill in the job title search field",
        "index": 3,
        "text": "software engineer"
      }
    }
```

```
  ]
}
```

Key design decisions:

- The Navigator gets a **list of interactive elements** with numeric indexes, not raw HTML. This abstracts away complex DOM structures and gives the LLM a clean, compact representation.
- It gets a **screenshot** if vision is enabled. This helps it understand visual layouts that aren't obvious from element text alone.
- It can emit **up to N actions per step** (configurable, default 10). For non-page-changing sequences (filling multiple form fields), it batches them for efficiency.

7.4 The Validator Agent

File: `chrome-extension/src/background/agent/validator.ts`

The Validator is a quality gate. After the Navigator completes all steps in the plan, the Validator checks:

- Did the final state match the original goal?
- Is the extracted data complete and correct?
- Should the Planner create a new plan to address remaining gaps?

```
// Validator output schema (simplified):
interface ValidationResult {
  is_valid: boolean;
  reason: string;
  answer: string | null; // The final answer to show the user
}
```

If `is_valid` is false, the Executor sends control back to the Planner for **replanning** — a new strategy incorporating what was learned.

---

8. LangChain.js — Orchestrating LLM Calls

8.1 Why LangChain?

Calling an LLM API directly is simple:

```
// Direct Anthropic API call:
const response = await anthropic.messages.create({
  model: "claude-haiku-4-5",
  max_tokens: 1024,
  messages: [{ role: "user", content: "Hello" }]
});
```

But an agent needs more:

- A unified interface across multiple providers (Anthropic, OpenAI, Gemini)
- Automatic retry on rate limits
- Token counting and context window management
- Structured output parsing with validation

- Easy swapping of models without changing application code

LangChain provides all of this. Dot uses it through the `@langchain/core` and `@langchain/anthropic` / `@langchain/openai` packages.

8.2 Creating a Chat Model

```
// chrome-extension/src/background/agent/helper.ts
import { ChatAnthropic } from '@langchain/anthropic';
import { ChatOpenAI } from '@langchain/openai';

// Factory function: creates the right LangChain model object
// based on which provider the user has configured:
export function createChatModel(providerConfig, agentModel) {
  switch (providerConfig.provider) {
    case 'anthropic':
      return new ChatAnthropic({
        model: agentModel.modelName,       // e.g., "claude-haiku-4-5"
        apiKey: providerConfig.apiKey,
        maxRetries: 3,                     // Retry on rate limits
        temperature: 0,                    // Deterministic outputs
      });

    case 'openai':
      return new ChatOpenAI({
        model: agentModel.modelName,
        apiKey: providerConfig.apiKey,
        temperature: 0,
      });

    // ... other providers
  }
}
```

8.3 Structured Output with Zod

LangChain's `.withStructuredOutput(schema)` method tells the LLM to return data matching a Zod schema. If the model returns invalid data, LangChain retries automatically.

```
import { z } from 'zod';

// Define what we expect the Navigator to return:
const NavigatorOutputSchema = z.object({
  current_state: z.object({
    evaluation_previous_goal: z.string(),
    memory: z.string(),
    next_goal: z.string(),
  }),
  action: z.array(z.record(z.unknown())),
});
```

```
// Bind the schema to the model:
const structuredNavigator = navigatorLLM.withStructuredOutput(NavigatorOutputSchema);

// Now when we invoke it, we get typed data back:
const result = await structuredNavigator.invoke(messages);
// result.current_state.memory is a string, guaranteed
// result.action is an array, guaranteed
```

8.4 Message History and Context Building

Each agent call builds a message array from the current context:

```
// How the Navigator builds its context:
const messages = [
  // 1. System prompt (agent's instructions and available tools):
  { role: 'system', content: navigatorSystemPrompt },

  // 2. Task description:
  { role: 'user', content: `Task: ${task}` },

  // 3. Current browser state:
  { role: 'user', content: buildBrowserStateMessage(browserState) },

  // 4. Screenshot (if vision enabled):
  { role: 'user', content: [{ type: 'image_url', image_url: screenshotBase64 }] },

  // 5. Previous step history:
  ...previousSteps.map(step => ({ role: 'assistant', content: JSON.stringify(step) })),
];

const decision = await structuredNavigator.invoke(messages);
```

---

9. The Executor — Coordinating Agent Execution

9.1 Role of the Executor

File: `chrome-extension/src/background/agent/executor.ts`

The Executor is the conductor. It:

1. Creates and holds the three agent instances
2. Runs the main agentic loop
3. Routes results between agents
4. Emits events to the side panel (via the Port)
5. Handles pausing, cancelling, and resuming

9.2 The Main Loop

```
// Simplified executor loop:
class Executor {
  async execute(): Promise<void> {
```

```
    this.emitEvent(Actors.SYSTEM, ExecutionState.TASK_START, this.task);

    let plannerResult = await this.runPlanner();
    let steps = 0;

    while (steps < this.maxSteps) {
      // 1. Run the Navigator for the current plan step:
      const navResult = await this.runNavigator(plannerResult.plan);

      if (navResult.isDone) {
        // 2. Validate the result:
        const validationResult = await this.runValidator(navResult.answer);

        if (validationResult.is_valid) {
          // SUCCESS: report back to the user
          this.context.finalAnswer = validationResult.answer;
          this.emitEvent(Actors.SYSTEM, ExecutionState.TASK_OK, validationResult.answer);
          return;
        } else {
          // FAIL: replan with new information
          plannerResult = await this.runPlanner(validationResult.reason);
        }
      }

      steps++;
    }

    // Ran out of steps:
    this.emitEvent(Actors.SYSTEM, ExecutionState.TASK_FAIL, "Max steps reached");
  }
}
```

9.3 Event Emission

Every significant moment in execution emits an event through the port:

```
// ExecutionState enum (types.ts):
enum ExecutionState {
  TASK_START = 'task_start',    // Task just began
  TASK_OK = 'task_ok',          // Task completed successfully
  TASK_FAIL = 'task_fail',      // Task failed
  TASK_CANCEL = 'task_cancel',  // User cancelled
  STEP_START = 'step_start',    // An agent started a step
  STEP_OK = 'step_ok',          // An agent completed a step
  ACT_START = 'act_start',      // An action started
  ACT_OK = 'act_ok',            // An action completed
  ACT_FAIL = 'act_fail',        // An action failed
}

// The side panel listens and updates the UI:
// "Planner is thinking..." → "Navigator is acting..." → "Done"
```

9.4 AgentContext — Shared State

```
// chrome-extension/src/background/agent/types.ts
class AgentContext {
  taskId: string;
  task: string;
  browserContext: BrowserContext;
  finalAnswer: string | null;
  hasClaimedTab: boolean;  // Has the agent opened its own tab yet?

  constructor(taskId, task, browserContext) {
    this.taskId = taskId;
    this.task = task;
    this.browserContext = browserContext;
    this.finalAnswer = null;
    this.hasClaimedTab = false; // New: agent starts on user's tab
  }
}
```

`hasClaimedTab` is a key Dot addition. On the first navigation, instead of replacing the user's current tab, the agent opens a **new tab**. This means you can give Dot a task and keep browsing in your current tab without interruption.

```
// In the go_to_url action implementation:
if (!this.context.hasClaimedTab) {
  // First navigation: open a new tab so user's tab is preserved
  this.context.hasClaimedTab = true;
  await this.context.browserContext.openTab(input.url);
} else {
  // Subsequent navigations: stay in the agent's own tab
  await this.context.browserContext.navigateTo(input.url);
}
```

---

10. Browser Control via CDP and Puppeteer

10.1 What is CDP?

Chrome DevTools Protocol (CDP) is a JSON-RPC-based protocol that Chrome exposes for debugging. When you open DevTools in Chrome, the DevTools UI communicates with the browser via CDP. Dot uses the same protocol to control tabs programmatically.

Chrome extensions have access to CDP via `chrome.debugger` API (which is why it's in the permissions list).

```
// Attach CDP to a tab:
await chrome.debugger.attach({ tabId }, '1.3');

// Send a CDP command (navigate):
await chrome.debugger.sendCommand({ tabId }, 'Page.navigate', {
  url: 'https://example.com'
});
```

```
// Send a CDP command (click at coordinates):
await chrome.debugger.sendCommand({ tabId }, 'Input.dispatchMouseEvent', {
  type: 'mousePressed', x: 100, y: 200, button: 'left'
});
```

10.2 The DOM Tree Extraction Pipeline

Before the Navigator can act, it needs to know what's on the page. Raw HTML is too verbose for an LLM context window (a typical page has megabytes of HTML). Dot extracts a **compressed, semantic element tree**:

```
// What the Navigator sees instead of raw HTML:
`[0]<div>Main content</div>
  [1]<input placeholder='Search jobs...'>
  [2]<button>Search</button>
  [3]<a href='/jobs/123'>Senior Engineer at Acme Corp</a>
  [4]<a href='/jobs/124'>Staff Engineer at Widgets Inc</a>`
```

Only **interactive elements** (inputs, buttons, links, selects) are included. Each gets a sequential index. The Navigator references elements by index:

```
{ "click_element": { "index": 2 } }
```

This representation is:

- **Compact**: 50–200 lines instead of thousands of HTML lines
- **Semantic**: Shows the user-facing text, not internal class names
- **Actionable**: Indexes directly correspond to clickable elements

10.3 The Content Script's Role

The DOM extraction logic runs in the **content script** (injected into every page) because the service worker cannot directly access a page's DOM. The content script runs `buildDomTree()`, which walks the DOM and returns the compact element representation, which is then forwarded to the service worker.

Part III — The Action System

11. What are Actions?

Actions are the **vocabulary** the agent uses to interact with the world. Every button click, text input, page navigation, data extraction, and API call is an action.

An action has three parts:

1. **Schema** — What inputs it accepts (defined with Zod)
2. **Implementation** — What it actually does (TypeScript function)
3. **Registration** — Where it's added to the agent's available tools

11.1 Action Flow

```
LLM decides: { "click_element": { "index": 42 } }
                          |
                          ▼
          Action Builder looks up "click_element"
                          |
                          ▼
          Validates input against Zod schema
                          |
                          ▼
          Executes: page.click(elementIndex=42)
                          |
                          ▼
          Returns: ActionResult { success: true }
                          |
                          ▼
          Event emitted: ACT_OK "click_element"
                          |
                          ▼
          Result added to context for next LLM call
```

---

12. Zod Schemas — Type-Safe Action Definitions

12.1 What is Zod?

Zod is a TypeScript-first schema validation library. You define a schema once, and Zod:

- Validates incoming data at runtime
- Infers TypeScript types automatically
- Produces human-readable error messages when validation fails

12.2 Defining an Action Schema

```ts
// chrome-extension/src/background/agent/actions/schemas.ts
import { z } from 'zod';

// Every action has: a name, a description (shown to the LLM),
// and a Zod schema defining its inputs.
export interface ActionSchema {
  name: string;
  description: string;
  schema: z.ZodType;
}

// Example: the click_element action
export const clickElementActionSchema: ActionSchema = {
  name: 'click_element',

  // The description is crucial — the LLM reads it to decide
  // when and how to use this action:
  description: 'Click element by index',
```

```
  schema: z.object({
    // 'intent' appears on every action — it's a chain-of-thought
    // field that makes the LLM explain its reasoning before acting:
    intent: z.string().default('').describe('purpose of this action'),

    // The element index from the DOM tree:
    index: z.number().int().describe('index of the element'),

    // Optional xpath for disambiguation:
    xpath: z.string().nullable().optional().describe('xpath of the element'),
  }),
};

// The TypeScript type is automatically derived:
// type ClickInput = { intent: string; index: number; xpath?: string | null }
type ClickInput = z.infer<typeof clickElementActionSchema.schema>;
```

12.3 Why `intent` on every action?

The `intent` field is a **chain-of-thought** technique. By requiring the LLM to state its purpose before acting, you get:

1. Better action accuracy (the model commits to a goal before choosing how to achieve it)
2. Interpretable logs (you can read what the agent was thinking)
3. Reduced hallucination (the model is less likely to take contradictory actions)

---

13. The Action Builder — Registering Tools

File: `chrome-extension/src/background/agent/actions/builder.ts`

13.1 The Action Class

```
// Each action is a class wrapping a schema and an async handler:
class Action {
  schema: ActionSchema;
  handler: (input: unknown) => Promise<ActionResult>;

  constructor(handler, schema) {
    this.handler = handler;
    this.schema = schema;
  }

  async execute(input: unknown): Promise<ActionResult> {
    // Validate input against Zod schema first:
    const validated = this.schema.schema.parse(input);
    return this.handler(validated);
  }
}
```

13.2 ActionResult

```
// Every action returns an ActionResult:
class ActionResult {
  extractedContent?: string; // Data to add to agent memory
  error?: string;            // Error message if action failed
  includeInMemory: boolean;  // Should this appear in future context?

  constructor({ extractedContent, error, includeInMemory = false }) {
    this.extractedContent = extractedContent;
    this.error = error;
    this.includeInMemory = includeInMemory;
  }
}
```

13.3 Building the Action Registry

```
class ActionBuilder {
  private context: AgentContext;

  constructor(context: AgentContext) {
    this.context = context;
  }

  buildActions(): Action[] {
    const actions: Action[] = [];

    // --- Navigation ---
    const goToUrl = new Action(async (input) => {
      this.context.emitEvent(Actors.NAVIGATOR, ExecutionState.ACT_START, `go_to_url:
${input.url}`);

      // KEY DOT FEATURE: Open a new tab on first navigation
      // so the user's current tab is never disturbed.
      if (!this.context.hasClaimedTab) {
        this.context.hasClaimedTab = true;
        await this.context.browserContext.openTab(input.url);
      } else {
        await this.context.browserContext.navigateTo(input.url);
      }

      this.context.emitEvent(Actors.NAVIGATOR, ExecutionState.ACT_OK, input.url);
      return new ActionResult({ includeInMemory: false });
    }, goToUrlActionSchema);

    actions.push(goToUrl);

    // --- Click ---
    const clickElement = new Action(async (input) => {
      this.context.emitEvent(Actors.NAVIGATOR, ExecutionState.ACT_START, `click element
${input.index}`);
      const page = await this.context.browserContext.getCurrentPage();
```

```
      await page.clickElement(input.index, input.xpath);
      this.context.emitEvent(Actors.NAVIGATOR, ExecutionState.ACT_OK, `clicked
${input.index}`);
      return new ActionResult({ includeInMemory: false });
    }, clickElementActionSchema);

    actions.push(clickElement);

    // ... (all other actions follow the same pattern)

    return actions;
  }
}
```

---

14. Custom Dot Actions Reference

Dot adds several actions beyond the standard navigation set:

14.1 `get_profile_field`

```
// Reads a field from the user's stored profile.
// The agent uses this to fill forms without being told the user's name/email.
export const getProfileFieldActionSchema: ActionSchema = {
  name: 'get_profile_field',
  description: 'Read a field from the user profile (name, email, phone, skills,
experience...)',
  schema: z.object({
    field: z.string().describe('the profile field to read'),
  }),
};

// Implementation (in builder.ts):
const getProfileField = new Action(async (input) => {
  const profile = await profileStore.get();
  const value = profile[input.field as keyof ProfileData];
  const result = value ? JSON.stringify(value) : `Field "${input.field}" not set`;
  return new ActionResult({ extractedContent: result, includeInMemory: true });
}, getProfileFieldActionSchema);
```

14.2 `save_results`

```
// Persists extracted data to the results store for later review.
// When called, the side panel shows a "Results Saved" message.
export const saveResultsActionSchema: ActionSchema = {
  name: 'save_results',
  description: 'Save extracted data to the results store for later review',
  schema: z.object({
    type: z.enum(['job', 'research', 'extraction']),
    data: z.string().describe('extracted data as JSON or plain text'),
```

```
  }),
};
```

14.3 `human_interrupt`

```
// Pauses execution and shows a modal to the user.
// MANDATORY before any financial action.
export const humanInterruptActionSchema: ActionSchema = {
  name: 'human_interrupt',
  description: 'Pause execution and prompt the user for input before continuing',
  schema: z.object({
    reason: z.string().describe('why the interruption is needed'),
    url: z.string().describe('current URL where the interrupt is triggered'),
  }),
};

// When the executor sees this action, it:
// 1. Pauses the agentic loop
// 2. Sends a 'human_interrupt' event to the side panel
// 3. Waits for the user to click Resume or Cancel
// 4. Resumes or aborts accordingly
```

14.4 `fetch_url`

```
// Makes an HTTP request FROM the current page's context,
// inheriting its session cookies. This enables direct API access
// to authenticated endpoints (LinkedIn Voyager API, GitHub API, etc.)
export const fetchUrlActionSchema: ActionSchema = {
  name: 'fetch_url',
  description: 'Make an HTTP request inheriting the page session cookies',
  schema: z.object({
    url: z.string(),
    method: z.string().default('GET'),
    headers: z.record(z.string()).optional(),
    body: z.string().optional(),
  }),
};

// Implementation uses chrome.scripting.executeScript to run
// the fetch from inside the page context (same origin, same cookies):
const injected = await chrome.scripting.executeScript({
  target: { tabId },
  func: async (url, method, headers, body) => {
    const res = await fetch(url, {
      method,
      credentials: 'include',   // Include session cookies
      headers: { 'Accept': 'application/json', ...headers },
      body,
    });
```

```
    return { ok: res.ok, status: res.status, text: await res.text() };
  },
  args: [input.url, input.method, input.headers, input.body],
});
```

14.5 `open_ai_chat`

```
// Opens Gemini, Claude, ChatGPT, or DeepSeek in a new tab.
// The agent uses this for tasks that benefit from a large context
// AI (generating documents, synthesizing complex research, etc.)
export const openAiChatActionSchema: ActionSchema = {
  name: 'open_ai_chat',
  description: 'Open an AI chat assistant in a new tab',
  schema: z.object({
    provider: z.enum(['gemini', 'claude', 'chatgpt', 'deepseek']).default('gemini'),
  }),
};

const AI_CHAT_URLS = {
  gemini: 'https://gemini.google.com/app',
  claude: 'https://claude.ai/new',
  chatgpt: 'https://chat.openai.com/',
  deepseek: 'https://chat.deepseek.com/',
};

// Opens in a new tab so the agent's working tab is unaffected
```

Part IV — The Storage Layer

15. Chrome Storage API

15.1 Types of Chrome Storage

Chrome extensions have access to several storage types:

API	Scope	Persists	Use in Dot
`chrome.storage.local`	Extension	Until cleared	All user data (profiles, history, results, watches)
`chrome.storage.session`	Browser session	Until browser restart	Context menu pending text, scheduled task queue
`chrome.storage.sync`	Synced across devices	Until cleared	Not used (sensitive data shouldn't sync)

15.2 Basic Usage

```
// Write:
await chrome.storage.local.set({ 'my-key': { name: 'Juan', email: 'juan@example.com' } });

// Read:
const result = await chrome.storage.local.get('my-key');
console.log(result['my-key']); // { name: 'Juan', ... }

// Listen for changes:
chrome.storage.onChanged.addListener((changes, area) => {
  if (area === 'local' && changes['my-key']) {
    console.log('New value:', changes['my-key'].newValue);
  }
});
```

15.3 The Problem with Raw Storage

Using `chrome.storage.local.set/get` directly throughout the codebase has problems:

- No TypeScript types — everything is `any`
- Repeated boilerplate in every file
- No reactivity — React components don't know when data changes
- No default values — reading a non-existent key returns `undefined`

The `createStorage` pattern solves all of these.

---

16. The createStorage Pattern

File: `packages/storage/lib/base/base.ts`

16.1 The Factory Function

```
// createStorage is a generic factory that wraps chrome.storage
// with types, defaults, and reactivity:
function createStorage<T>(
  key: string,           // Storage key
  defaultValue: T,       // What to return if key doesn't exist
  options: {
    storageEnum: StorageEnum; // 'local' | 'session' | 'sync'
    liveUpdate: boolean;      // Should React components re-render on change?
  }
): BaseStorage<T> {
  // ...
}
```

16.2 The BaseStorage Interface

```
// Every store has these methods:
interface BaseStorage<T> {
  get: () => Promise<T>;                    // Read current value
```

```
  set: (value: T | ((prev: T) => T)) => Promise<void>; // Write (supports updater fn)
  subscribe: (callback: (value: T) => void) => () => void; // React reactivity hook
}
```

16.3 Using createStorage in Practice

```
// packages/storage/lib/resultsStore.ts
import { createStorage } from './base/base';
import { StorageEnum } from './base/enums';

// 1. Define the data shape:
export interface SavedResult {
  id: string;
  type: 'job' | 'research' | 'extraction';
  data: string;
  source: string;
  timestamp: number;
}

// 2. Create the underlying storage:
const storage = createStorage<SavedResult[]>('results-data', [], {
  storageEnum: StorageEnum.Local,
  liveUpdate: true, // React re-renders when data changes
});

// 3. Build a typed store on top:
export const resultsStore = {
  ...storage,

  // Add domain-specific methods:
  async addResult(result: Omit<SavedResult, 'id' | 'timestamp'>): Promise<SavedResult> {
    const newResult: SavedResult = {
      ...result,
      id: `${Date.now()}-${Math.random().toString(36).slice(2)}`,
      timestamp: Date.now(),
    };

    // Use the functional updater form to safely append to the array:
    await storage.set(prev => [...prev, newResult]);

    return newResult;
  },

  async getResults(type?: SavedResult['type']): Promise<SavedResult[]> {
    const all = await storage.get();
    return type ? all.filter(r => r.type === type) : all;
  },

  async clearResults(): Promise<void> {
    await storage.set([]);
```

```
  },
};
```

16.4 Using a Store in a React Component

```
// React hook for reactive store access:
import { useStorage } from '@extension/storage';
import { resultsStore } from '@extension/storage';

function ResultsBadge() {
  // useStorage subscribes to the store.
  // When resultsStore changes (new result saved by the agent),
  // this component automatically re-renders:
  const results = useStorage(resultsStore);

  return <span>{results.length} saved results</span>;
}
```

---

17. All Stores

17.1 Profile Store

```
// packages/storage/lib/profileStore.ts
export interface ProfileData {
  name: string;
  email: string;
  phone: string;
  location: string;
  skills: string[];
  education: Array<{ school: string; degree: string; year: string }>;
  experience: Array<{ company: string; role: string; duration: string; description: string
}>;
  links: { linkedin?: string; github?: string; portfolio?: string };
}
```

The Profile Store holds the user's personal data. When the agent calls `get_profile_field('email')`, it reads from this store and injects the value into the form. **This data never leaves your device** except in LLM prompts (where it's used to fill forms).

17.2 Results Store

```
// packages/storage/lib/resultsStore.ts
export interface SavedResult {
  id: string;
  type: 'job' | 'research' | 'extraction';
  data: string;           // JSON string or plain text
  source: string;         // URL where data was extracted
```

```
  timestamp: number;      // Unix ms
}
```

When the agent calls `save_results`, data lands here. The Results view in the side panel reads from this store and renders `ResultsCard` components.

17.3 Upload Store

```
// packages/storage/lib/uploadStore.ts
export interface UploadedFile {
  id: string;
  name: string;
  type: 'md';             // All uploads are stored as markdown
  content: string;        // The parsed text content
  size: number;           // Original file size in bytes
  timestamp: number;
}
```

Files attached to a task are stored here. The agent calls `list_uploaded_files()` to discover them, then `read_uploaded_file(fileId)` to read their content, injecting it into its context.

17.4 Watch Store

```
// packages/storage/lib/watchStore.ts
export interface WatchConfig {
  id: string;
  url: string;                  // URL to monitor
  label: string;                // Human-readable name
  intervalMinutes: number;      // How often to check
  lastSnapshot: string | null;  // Last fetched text content
  lastChecked: number | null;   // Unix ms of last check
  active: boolean;
  createdAt: number;
}
```

Each WatchConfig corresponds to a `chrome.alarms` entry named `dot-watch-${id}`. When the alarm fires, the background fetches the URL, extracts its text content, and compares it to `lastSnapshot`.

17.5 Scheduled Task Store

```
// packages/storage/lib/scheduledTaskStore.ts
export interface ScheduledTask {
  id: string;
  label: string;
  taskDescription: string;    // The task instruction for the agent
  intervalMinutes: number;
  nextRunAt: number | null;   // Unix ms of next scheduled run
  lastRunAt: number | null;
  active: boolean;
```

```
  createdAt: number;
}
```

Each active task has a `chrome.alarms` entry named `dot-task-${id}`. When it fires, the task description is stored in `chrome.storage.session` as `dotPendingScheduledTask`, and a notification appears. When the side panel is opened, it detects the pending task and auto-runs it.

Part V — The Frontend

18. React Inside a Chrome Extension

18.1 How Vite Bundles the Side Panel

The side panel is a standard React application, bundled by Vite into static files placed in `dist/side-panel/`. Chrome loads it like a local webpage.

```
// pages/side-panel/src/main.tsx
import React from 'react';
import { createRoot } from 'react-dom/client';
import SidePanel from './SidePanel';

// Standard React mounting — no Chrome-specific code here:
const root = createRoot(document.getElementById('app')!);
root.render(<SidePanel />);
```

```
<!-- pages/side-panel/index.html -->
<!DOCTYPE html>
<html>
  <head>
    <!-- Google Fonts for the design system: -->
    <link href="https://fonts.googleapis.com/css2?
family=Manrope:wght@300;400;600;700&display=swap" rel="stylesheet">
  </head>
  <body>
    <div id="app"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
```

18.2 Accessing Chrome APIs from React

React components can call Chrome APIs directly because the side panel runs in a Chrome extension context:

```
// Get the current tab in a React component:
const tabs = await chrome.tabs.query({ active: true, currentWindow: true });
const currentTab = tabs[0];

// Read from session storage:
```

```
const result = await chrome.storage.session.get('dotContextMenuPending');

// Open the options page:
chrome.runtime.openOptionsPage();
```

No special wrappers needed — `chrome` is a global in extension contexts.

---

19. Component Architecture

```
SidePanel.tsx (root — state machine)
├── TopBar.tsx           (branding, dark mode toggle)
├── BootScreen.tsx       (initial power-on screen)
├── WorkflowPicker.tsx   (pre-built task templates)
├── MessageList.tsx      (chat history)
│   ├── AgentMessage.tsx (styled AI bubble)
│   ├── UserMessage.tsx  (styled user bubble)
│   ├── StatusRow.tsx    (execution status line)
│   └── ResultsCard.tsx  (structured data display)
├── LiveStatusBar.tsx    (real-time "Navigator is acting...")
├── ChatInput.tsx        (text area, send, mic, file attach)
│   ├── FileUpload.tsx   (file picker, PDF parser)
│   └── FileChip.tsx     (attached file indicator)
├── ChatHistoryList.tsx  (past sessions browser)
├── ResultsList.tsx      (saved results + diff mode)
├── WatchList.tsx        (web monitoring configuration)
└── ScheduledTaskList.tsx (scheduled task management)
```

19.1 Component Design Principles

Single responsibility: Each component owns one concern. `ChatInput` handles user input; `MessageList` renders history; `LiveStatusBar` shows live status. They don't cross concerns.

Props down, events up: Data flows from `SidePanel` (which owns all state) down to children via props. Children report user actions back up via callback props (`onSendMessage` , `onClose` , etc.).

No component-level storage: Components read from Chrome storage only through the `useStorage` hook (which subscribes via `createStorage` 's `subscribe` method). They never call `chrome.storage.local.get` directly.

---

20. The Design System

20.1 CSS Variables

```
/* pages/side-panel/src/index.css */

:root {
  /* The three-tone palette: */
  --bg:      #EBEBEB;   /* Ground — page background */
  --text:    #1A1A1A;   /* Ink — primary text */
  --accent:  #C45A2D;   /* Burnt amber — highlights, buttons */
```

```
  /* Derived tones: */
  --muted:   rgba(26,26,26,0.45);   /* Secondary text */
  --line:    rgba(0,0,0,0.09);      /* Borders, dividers */
  --surface: rgba(255,255,255,0.6); /* Card backgrounds */

  /* Glassmorphism: */
  --glass:   rgba(255,255,255,0.50);
  --glass-b: rgba(255,255,255,0.75);
}

/* Dark mode — triggered by data-theme="dark" on the root div: */
[data-theme="dark"] {
  --bg:      #0C0C0C;
  --text:    #E8E6E1;
  --accent:  #D4714A;
  --muted:   rgba(232,230,225,0.45);
  --line:    rgba(255,255,255,0.07);
  --surface: rgba(24,24,22,0.60);
  --glass:   rgba(24,24,22,0.60);
  --glass-b: rgba(255,255,255,0.05);
}
```

20.2 Typography

```
/* Three font families with semantic roles: */

.font-display {
  font-family: 'Cormorant Garamond', Georgia, serif;
  /* Used for the "DOT" brand name — editorial, authoritative */
}

body {
  font-family: 'Manrope', system-ui, sans-serif;
  /* Used for all body text — clean, modern, readable */
}

.label-mono {
  font-family: 'Courier New', Courier, monospace;
  font-size: 9px;
  text-transform: uppercase;
  letter-spacing: 0.12em;
  /* Used for status labels, timestamps, badges — technical, precise */
}
```

20.3 Glassmorphism Implementation

```
/* The glass card effect: */
.glass-card {
  background: var(--glass);
```

```
  backdrop-filter: blur(20px) saturate(140%);
  -webkit-backdrop-filter: blur(20px) saturate(140%);
  border: 1px solid var(--line);
  border-radius: 8px;
}

/* Why does this look good?
   backdrop-filter: blur() processes the pixels BEHIND the element,
   creating the frosted glass look. saturate() makes the blurred
   background colors more vivid. The semi-transparent background
   (var(--glass)) lets the blur show through. */
```

---

21. Long-Lived Port Communication

21.1 Why Long-Lived?

A task can run for minutes. Using `chrome.runtime.sendMessage` for each event would create hundreds of independent round-trips. A **Port** (like a WebSocket) is opened once and stays open for the entire task duration.

21.2 The Connection Setup

```
// pages/side-panel/src/SidePanel.tsx

const setupConnection = useCallback(() => {
  // Don't create a second connection if one already exists:
  if (portRef.current) return;

  // Open the port:
  portRef.current = chrome.runtime.connect({ name: 'side-panel-connection' });

  // Listen for messages (events from the executor):
  portRef.current.onMessage.addListener((message) => {
    if (message.type === 'execution') {
      handleTaskState(message); // Update UI based on event
    }
  });

  // Handle disconnection (side panel closed, service worker restarted):
  portRef.current.onDisconnect.addListener(() => {
    portRef.current = null;
    setInputEnabled(true);
    setShowStopButton(false);
  });

  // Send periodic heartbeats so the service worker knows the panel is alive:
  // (Service workers sleep after inactivity; heartbeats keep them awake)
  heartbeatIntervalRef.current = setInterval(() => {
    portRef.current?.postMessage({ type: 'heartbeat' });
  }, 25000); // Every 25 seconds
}, [handleTaskState]);
```

21.3 The Heartbeat Mechanism

Chrome MV3 service workers go idle after ~30 seconds of inactivity, interrupting long tasks. The side panel sends a `heartbeat` message every 25 seconds to keep the service worker active.

```
// In background/index.ts:
case 'heartbeat':
  port.postMessage({ type: 'heartbeat_ack' });
  break;
  // The simple act of receiving and responding to a message
  // resets Chrome's idle timer, keeping the service worker alive.
```

---

22. SidePanel State Machine

`SidePanel.tsx` is the largest file in the project. It manages all UI state as a flat set of `useState` hooks that together describe which view is shown and what data it contains.

22.1 View State

```
// Which view is currently shown:
const [showHistory, setShowHistory] = useState(false);
const [showResults, setShowResults] = useState(false);
const [showWatches, setShowWatches] = useState(false);
const [showSchedules, setShowSchedules] = useState(false);
// All false → main chat view
```

22.2 Task Lifecycle State

```
const [inputEnabled, setInputEnabled] = useState(true);
// true = user can type, false = agent is running

const [showStopButton, setShowStopButton] = useState(false);
// Shows a Stop button during active tasks

const [isFollowUpMode, setIsFollowUpMode] = useState(false);
// true = previous task is done, new messages are follow-ups
// (sent as follow_up_task instead of new_task)

const [isHistoricalSession, setIsHistoricalSession] = useState(false);
// true = user loaded a past session, input blocked until "Continue" is clicked
```

22.3 Event Handling

The `handleTaskState` function is the central router for all execution events:

```
const handleTaskState = useCallback((event: AgentEvent) => {
  const { actor, state, data } = event;
```

```
  switch (actor) {
    case Actors.SYSTEM:
      switch (state) {
        case ExecutionState.TASK_OK:
          // Batch mode: chain to next URL if more remain
          const bq = batchQueueRef.current;
          if (bq && bq.index + 1 < bq.urls.length) {
            // Fire next URL in the batch
            const nextIdx = bq.index + 1;
            batchQueueRef.current = { ...bq, index: nextIdx };
            portRef.current?.postMessage({
              type: 'follow_up_task',
              task: `${bq.urls[nextIdx]}\n${bq.instruction}`,
              // ...
            });
          } else {
            // Task truly complete: show final answer
            batchQueueRef.current = null;
            setInputEnabled(true);
            setIsFollowUpMode(true);
            // Show the agent's completion message (if it's not just a UUID):
            const isUUID = /^[0-9a-f-]{36}$/i.test(data?.details ?? '');
            if (data?.details && !isUUID) {
              appendMessage({ actor, content: data.details, timestamp: Date.now() });
            }
          }
          break;

        case ExecutionState.TASK_FAIL:
          setInputEnabled(true);
          setShowStopButton(false);
          break;
      }
      break;

    case Actors.NAVIGATOR:
      switch (state) {
        case ExecutionState.ACT_START:
          // Update the live status bar:
          setLiveStatus({ actor: 'NAVIGATOR', text: data?.details ?? 'Acting...' });
          break;
        // ...
      }
      break;
  }
}, [appendMessage]);
```

Part VI — Features Deep Dive

23. Batch URL Mode

23.1 The Problem

You have 5 job listings to check. Without batch mode, you'd have to:

1. Send task for URL 1, wait for completion
2. Send task for URL 2, wait for completion
3. Repeat 5 times

23.2 The Solution

Paste multiple URLs with one instruction:

```
https://company-a.com/careers/123
https://company-b.com/jobs/456
https://company-c.com/positions/789

Check each listing and tell me: required experience level, tech stack, and remote policy
```

23.3 How It Works

```
// Detect batch mode input:
function parseBatchInput(text: string): { urls: string[]; instruction: string } | null {
  const lines = text.split('\n').map(l => l.trim()).filter(Boolean);
  const urlPattern = /^https?:\/\/\S+$/;

  // Separate URL lines from instruction lines:
  const urls = lines.filter(l => urlPattern.test(l));
  const instructionLines = lines.filter(l => !urlPattern.test(l));

  // Need at least 2 URLs and 1 instruction:
  if (urls.length >= 2 && instructionLines.length >= 1) {
    return { urls, instruction: instructionLines.join(' ') };
  }
  return null;
}

// On task completion (TASK_OK), the side panel chains the next URL:
if (bq && bq.index + 1 < bq.urls.length) {
  const nextIdx = bq.index + 1;
  batchQueueRef.current = { ...bq, index: nextIdx };

  // Send as a follow_up_task (reuses the same executor session):
  portRef.current.postMessage({
    type: 'follow_up_task',
    task: `${bq.urls[nextIdx]}\n${bq.instruction}`,
    taskId: sessionIdRef.current,
    tabId: activeTabIdRef.current,
  });
}
```

The key insight: `follow_up_task` reuses the existing executor and browser context. The agent doesn't start fresh — it continues in the same session with the same memory, which is more efficient and produces consistent output format.

---

24. Web Watches and Chrome Alarms

24.1 Chrome Alarms API

`chrome.alarms` is a service-worker-safe scheduling API. Unlike `setInterval` (which stops when the service worker goes idle), alarms persist and wake the service worker at the scheduled time.

```
// Create an alarm:
chrome.alarms.create('dot-watch-abc123', {
  delayInMinutes: 60  // First fire in 60 minutes
});

// Listen for alarms:
chrome.alarms.onAlarm.addListener(async (alarm) => {
  if (alarm.name.startsWith('dot-watch-')) {
    const watchId = alarm.name.replace('dot-watch-', '');
    await checkWatch(watchId);
  }
});

// Clear an alarm:
chrome.alarms.clear('dot-watch-abc123');
```

24.2 The checkWatch Function

```
async function checkWatch(watchId: string) {
  // Load current watch config:
  const watches = await watchStore.getAll();
  const watch = watches.find(w => w.id === watchId);
  if (!watch || !watch.active) return;

  try {
    // Fetch the page (from the service worker, not a tab):
    const response = await fetch(watch.url);
    const html = await response.text();

    // Extract meaningful text (strip HTML tags, scripts, styles):
    const text = html
      .replace(/<script[\s\S]*?<\/script>/gi, '')   // Remove scripts
      .replace(/<style[\s\S]*?<\/style>/gi, '')      // Remove styles
      .replace(/<[^>]+>/g, ' ')                      // Strip all tags
      .replace(/\s+/g, ' ')                          // Normalize whitespace
      .trim()
      .slice(0, 10000);                              // Cap at 10k chars

    // Compare with the stored snapshot:
```

```
    if (watch.lastSnapshot !== null && text !== watch.lastSnapshot) {
      // PAGE CHANGED! Fire a notification:
      chrome.notifications.create(`watch-changed-${watchId}-${Date.now()}`, {
        type: 'basic',
        iconUrl: 'icon-128.png',
        title: 'Dot — Page Changed',
        message: `"${watch.label}" has new content`,
      });
    }

    // Update the stored snapshot:
    await watchStore.update(watchId, {
      lastSnapshot: text,
      lastChecked: Date.now(),
    });

  } catch (e) {
    console.error('Watch check failed:', e);
  }

  // Re-arm the alarm for the next check:
  // (We don't use 'periodInMinutes' because we want manual re-arming
  // for robustness — if a check fails, we still reschedule)
  chrome.alarms.create(`dot-watch-${watchId}`, {
    delayInMinutes: watch.intervalMinutes
  });
}
```

24.3 Alarm Persistence Across Browser Restarts

Chrome clears alarms when the browser restarts (unless you use the `when` property with a future timestamp). Dot re-registers all active alarms on `onInstalled` and `onStartup`:

```
chrome.runtime.onStartup.addListener(() => {
  registerActiveAlarms();
});

async function registerActiveAlarms() {
  const watches = await watchStore.getAll();
  for (const watch of watches.filter(w => w.active)) {
    const existing = await chrome.alarms.get(`dot-watch-${watch.id}`);
    if (!existing) {
      // Alarm was lost — recreate it:
      chrome.alarms.create(`dot-watch-${watch.id}`, {
        delayInMinutes: watch.intervalMinutes
      });
    }
  }
}
```

---

25. Scheduled Tasks

Scheduled tasks follow the same alarm pattern as web watches, but instead of fetching a URL, they trigger the agent.

25.1 The Flow

```
Alarm fires (dot-task-abc123)
  │
  ▼
triggerScheduledTask('abc123')
  │
  ▼
Load task from scheduledTaskStore
  │
  ▼
Store task in chrome.storage.session as 'dotPendingScheduledTask'
  │
  ▼
Show notification: "Task X is ready to run"
  │
  ▼
Update task's nextRunAt and lastRunAt
  │
  ▼
Re-arm alarm for next interval
  │
  ▼ (User opens side panel or clicks notification)
  │
  ▼
SidePanel.tsx mounts → checks chrome.storage.session
  │
  ▼
Finds dotPendingScheduledTask → calls handleSendMessage(task.taskDescription)
  │
  ▼
Agent runs the task
```

25.2 Why Session Storage for the Pending Task?

`chrome.storage.session` is the bridge between the service worker (where the alarm fires) and the side panel (where the task runs). It:

- Persists for the browser session lifetime
- Is accessible from both contexts
- Is automatically cleared on browser restart
- Is separate from `local` storage (won't pollute the persistent store)

---

26. Context Menu Integration

26.1 Registration

```
// background/index.ts — runs once when extension is installed:
chrome.runtime.onInstalled.addListener(() => {
  chrome.contextMenus.create({
    id: 'dot-ask-about',          // Unique identifier
    title: 'Ask Dot about "%s"',  // %s is replaced with selected text
    contexts: ['selection'],       // Only shows when text is selected
  });
});
```

26.2 The Click Handler

```
chrome.contextMenus.onClicked.addListener((info, tab) => {
  if (info.menuItemId !== 'dot-ask-about' || !info.selectionText) return;

  // Store the selected text in session storage:
  chrome.storage.session.set({
    dotContextMenuPending: {
      text: info.selectionText,
      pageUrl: info.pageUrl ?? '',
      timestamp: Date.now(),
    },
  });

  // Open the side panel programmatically:
  // (This is a user gesture handler, so we're allowed to open UI)
  if (tab?.id) {
    chrome.sidePanel.open({ tabId: tab.id });
  }
});
```

26.3 The Side Panel Pre-fill

```
// SidePanel.tsx — runs once on mount:
useEffect(() => {
  // Slight delay to let ChatInput mount and register its setter:
  const timer = setTimeout(async () => {
    const result = await chrome.storage.session.get('dotContextMenuPending');
    const pending = result.dotContextMenuPending;

    if (pending?.text && setInputTextRef.current) {
      // Build a natural language query from the selection:
      const prefill = `"${pending.text.slice(0, 300)}"${
        pending.pageUrl ? `\n(from ${pending.pageUrl})` : ''
      }`;

      setInputTextRef.current(prefill);  // Set the input text
      await chrome.storage.session.remove('dotContextMenuPending'); // Consume it
    }
  }, 400);
```

```
  return () => clearTimeout(timer);
}, []);
```

---

27. Result Diffing

27.1 The Problem

You ran a job extraction last Monday and again today. Which jobs are new? Which disappeared? Which changed their requirements? Manually comparing two JSON blobs is painful.

27.2 The Algorithm

```
// For JSON arrays (most common case):
function arrayDiff(a: unknown[], b: unknown[]) {
  // First, determine a stable key for each item.
  // We look for common identity fields in priority order:
  function getItemKey(item: unknown): string {
    if (typeof item !== 'object' || !item) return JSON.stringify(item);
    const obj = item as Record<string, unknown>;
    for (const f of ['id', 'url', 'title', 'name', 'link']) {
      if (obj[f]) return String(obj[f]);  // Use first found identity field
    }
    return JSON.stringify(item);  // Fall back to full serialization
  }

  // Build maps for O(1) lookup:
  const aMap = new Map(a.map(x => [getItemKey(x), x]));
  const bMap = new Map(b.map(x => [getItemKey(x), x]));

  const added   = b.filter(x => !aMap.has(getItemKey(x)));
  const removed = a.filter(x => !bMap.has(getItemKey(x)));
  const changed = b.filter(x => {
    const k = getItemKey(x);
    // In B but also in A, but with different values:
    return aMap.has(k) && JSON.stringify(aMap.get(k)) !== JSON.stringify(x);
  });

  return { added, removed, changed };
}
```

27.3 For Plain Text

```
// Line-by-line diff (set-based, not LCS-based for simplicity):
function lineDiff(a: string, b: string): DiffLine[] {
  const aLines = a.split('\n');
  const bLines = b.split('\n');
  const aSet = new Set(aLines);
  const bSet = new Set(bLines);
```

```
  const result: DiffLine[] = [];
  const seen = new Set<string>();

  // Lines in A: mark as removed if not in B, same if in B:
  for (const line of aLines) {
    if (!bSet.has(line))            result.push({ kind: 'removed', text: line });
    else if (!seen.has(line)) {
      seen.add(line);
      result.push({ kind: 'same', text: line });
    }
  }

  // Lines in B but not in A: added:
  for (const line of bLines) {
    if (!aSet.has(line)) result.push({ kind: 'added', text: line });
  }

  return result;
}
```

---

28. File Upload and PDF Processing

28.1 Architecture

```
User drops file on ChatInput
  │
  ▼
FileUpload.tsx receives File object
  │
  ├── .md file → file.text() → plain UTF-8 string
  │
  └── .pdf file → pdfjs-dist → extract text from each page → markdown string
  │
  ▼
uploadStore.addFile(content)
  │
  ▼
FileChip shown in chat input area
  │
  ▼ (user sends task)
  │
Agent calls list_uploaded_files()
→ [{ id: "abc", name: "resume.md", type: "md" }]

Agent calls read_uploaded_file("abc")
→ "# John Doe\n\nSoftware Engineer..."

Agent injects content into its task context
```

28.2 PDF Parsing with pdfjs-dist

```
// pages/side-panel/src/components/FileUpload.tsx
import * as pdfjsLib from 'pdfjs-dist';

// The PDF worker must be configured to run in a separate thread:
// (Heavy computation — you don't want it blocking the UI thread)
pdfjsLib.GlobalWorkerOptions.workerSrc =
  new URL('pdfjs-dist/build/pdf.worker.mjs', import.meta.url).toString();

async function pdfToMarkdown(file: File): Promise<string> {
  // Convert File to ArrayBuffer (raw bytes):
  const arrayBuffer = await file.arrayBuffer();

  // Load the PDF:
  const pdf = await pdfjsLib.getDocument({ data: arrayBuffer }).promise;

  const baseName = file.name.replace(/\.pdf$/i, '');
  const parts: string[] = [`# ${baseName}\n`];  // Use filename as h1

  // Extract text from each page:
  for (let i = 1; i <= pdf.numPages; i++) {
    const page = await pdf.getPage(i);
    const content = await page.getTextContent();

    // content.items is an array of text spans with coordinates.
    // We join them, handling the TextMarkedContent union type:
    const pageText = content.items
      .map(item => ('str' in item ? item.str : ''))  // TypeScript union guard
      .join(' ')
      .replace(/ {2,}/g, ' ')   // Normalize multiple spaces
      .trim();

    if (pageText) {
      // Add page separator for multi-page documents:
      if (pdf.numPages > 1) parts.push(`\n---\n_Page ${i}_\n`);
      parts.push(pageText);
    }
  }

  return parts.join('\n');
}
```

---

29. Financial Guardrails

29.1 The Threat Model

Without safeguards, an AI agent running in your browser could be manipulated into:

- Completing an online purchase you didn't intend
- Submitting a payment form with your stored card details
- Transferring money via an online banking interface

This is called **prompt injection** — a malicious website embeds hidden instructions in its page content that the agent reads and executes.

29.2 Defense-in-Depth

Dot implements guardrails at three layers:

Layer 1: Common security rules (shared across all agents)

```
// chrome-extension/src/background/agent/prompts/templates/common.ts
export const commonSecurityRules = `
## FINANCIAL GUARDRAILS — MANDATORY, NO EXCEPTIONS

Before clicking ANY element whose text matches: Buy, Pay now, Purchase,
Place order, Confirm order, Subscribe, Checkout, Transfer, Send money,
Confirm payment, Authorize — you MUST call human_interrupt first.

Before submitting ANY form containing: credit card number, CVV, expiry,
bank account, routing number, IBAN — call human_interrupt first.

If the current URL contains: checkout, payment, billing, cart/confirm,
transfer, bank, financial — pause and call human_interrupt before any
confirming action.

THIS RULE OVERRIDES ALL OTHER RULES. Even if the plan says to proceed.
`;
```

Layer 2: Planner-level safeguards

```
// planner.ts system prompt includes:
`# FINANCIAL SAFETY
If any step in the plan involves a financial action, you MUST insert
a human_interrupt step immediately before it. This step must describe:
- What financial action is about to occur
- The amount involved (if visible)
- What the user should verify before proceeding`
```

Layer 3: Navigator-level hard stop

```
// navigator.ts system prompt includes:
`15. Financial Safety (MANDATORY — no exceptions):
- Before clicking Buy, Pay now, Purchase, Place order, Confirm order,
  Complete purchase, Subscribe, Checkout, Transfer, Send money,
  Confirm payment, Authorize or any close equivalent — call human_interrupt first.
- This rule overrides all other rules. Even if the plan says to proceed.`
```

The `human_interrupt` action pauses the executor, shows a modal in the side panel, and waits for explicit user approval before continuing. The user can also cancel entirely.

---

30. AI Consultation via open_ai_chat

30.1 When Does the Agent Use This?

The planner is instructed to use `open_ai_chat` for:

- Generating complex documents (cover letters, reports, proposals)
- Synthesizing research from multiple sources into a structured output
- Writing or debugging code
- Any task requiring creativity or extended reasoning beyond simple web navigation

30.2 The Structured Prompt Pattern

When the agent opens an AI chat, it types a structured prompt using this template:

```
[Context]: Brief description of the broader task and relevant background.
[Request]: The specific, precise thing needed — explicit about format, length, style.
[Constraints]: Important rules, limits, or things to avoid.
[Output format]: Exactly how the response should be structured.
```

For example, if a user asks Dot to apply for a job, the agent might:

1. Call `get_profile_field('experience')` and `read_uploaded_file(resumeId)`
2. Open Gemini with `open_ai_chat`
3. Type: "*[Context]: I'm applying for a Senior Backend Engineer role at Acme Corp (Python, distributed systems). [Request]: Write a tailored cover letter using this experience: [paste]. [Constraints]: 3 paragraphs max, formal tone, no clichés. [Output format]: Plain text, ready to paste.*"
4. Extract the response
5. Use it to fill the application form

Part VII — Building from Scratch

31. From an Empty Folder to a Running Extension

This section walks you through building a minimal Chrome extension with a service worker, a side panel, and a simple "echo" agent — no prior extension experience required.

31.1 Create the Workspace

```
mkdir my-agent && cd my-agent
npm init -y
```

31.2 The Manifest

```
// manifest.json
{
  "manifest_version": 3,
  "name": "My Agent",
  "version": "0.1.0",
```

```
  "permissions": ["storage", "tabs", "sidePanel"],
  "background": {
    "service_worker": "background.js"
  },
  "side_panel": {
    "default_path": "panel.html"
  },
  "action": { "default_icon": "icon.png" }
}
```

31.3 The Service Worker

```
// background.js

// Open the side panel when the extension icon is clicked:
chrome.sidePanel.setPanelBehavior({ openPanelOnActionClick: true });

// Listen for messages from the panel:
chrome.runtime.onConnect.addListener(port => {
  if (port.name !== 'my-agent') return;

  port.onMessage.addListener(message => {
    if (message.type === 'task') {
      // Echo back after 1 second (simulating agent work):
      setTimeout(() => {
        port.postMessage({
          type: 'result',
          text: `I would execute: "${message.task}"`
        });
      }, 1000);
    }
  });
});
```

31.4 The Side Panel HTML

```
<!-- panel.html -->
<!DOCTYPE html>
<html>
<body>
  <input id="input" placeholder="Enter a task..." />
  <button id="send">Send</button>
  <div id="output"></div>

  <script>
    const port = chrome.runtime.connect({ name: 'my-agent' });
    const output = document.getElementById('output');

    port.onMessage.addListener(msg => {
```

```
      if (msg.type === 'result') {
        output.textContent = msg.text;
      }
    });

    document.getElementById('send').onclick = () => {
      const task = document.getElementById('input').value;
      port.postMessage({ type: 'task', task });
      output.textContent = 'Working...';
    };
  </script>
</body>
</html>
```

Load this as an unpacked extension. You now have a working extension with a side panel and background communication. This is the foundation Dot builds on.

---

32. Writing Your First Custom Action

Let's add a `get_current_time` action that the agent can call:

Step 1: Define the schema

```
// In schemas.ts, add:
export const getCurrentTimeActionSchema: ActionSchema = {
  name: 'get_current_time',
  description: 'Get the current date and time',
  schema: z.object({
    timezone: z.string().default('UTC').describe('IANA timezone name, e.g.
America/Toronto'),
  }),
};
```

Step 2: Implement the action

```
// In builder.ts, inside buildActions():
const getCurrentTime = new Action(async (input) => {
  this.context.emitEvent(Actors.NAVIGATOR, ExecutionState.ACT_START, 'get_current_time');

  const now = new Date();
  const formatted = now.toLocaleString('en-US', { timeZone: input.timezone });

  this.context.emitEvent(Actors.NAVIGATOR, ExecutionState.ACT_OK, formatted);

  return new ActionResult({
    extractedContent: `Current time in ${input.timezone}: ${formatted}`,
    includeInMemory: true,  // Agent will remember this in future steps
  });
}, getCurrentTimeActionSchema);
```

```
actions.push(getCurrentTime);
```

Step 3: Import and register the schema

```
// In schemas.ts exports, and in builder.ts imports:
import { getCurrentTimeActionSchema } from './schemas';
```

That's it. The agent now has access to `get_current_time` and will use it when it needs the current time.

---

33. Writing Your First Store

Let's add a `notesStore` for the agent to save arbitrary notes:

Step 1: Define the interface and storage

```
// packages/storage/lib/notesStore.ts
import { StorageEnum } from './base/enums';
import { createStorage } from './base/base';
import type { BaseStorage } from './base/types';

// The shape of a note:
export interface Note {
  id: string;
  content: string;
  createdAt: number;
}

// The store type (BaseStorage + custom methods):
export type NotesStorage = BaseStorage<Note[]> & {
  addNote: (content: string) => Promise<Note>;
  clearNotes: () => Promise<void>;
};

// Create the underlying storage:
const storage = createStorage<Note[]>('dot-notes', [], {
  storageEnum: StorageEnum.Local,
  liveUpdate: true,
});

// Export the store:
export const notesStore: NotesStorage = {
  ...storage,

  async addNote(content: string): Promise<Note> {
    const note: Note = {
      id: `note-${Date.now()}-${Math.random().toString(36).slice(2)}`,
      content,
      createdAt: Date.now(),
```

```
    };
    await storage.set(prev => [...prev, note]);
    return note;
  },

  async clearNotes(): Promise<void> {
    await storage.set([]);
  },
};
```

Step 2: Export from index

```
// packages/storage/lib/index.ts — add:
export * from './notesStore';
```

Step 3: Use in an action

```
// In builder.ts:
import { notesStore } from '@extension/storage';

const saveNote = new Action(async (input) => {
  const note = await notesStore.addNote(input.content);
  return new ActionResult({
    extractedContent: `Note saved with id: ${note.id}`,
    includeInMemory: true,
  });
}, saveNoteActionSchema);
```

---

34. Writing Your First UI Component

Let's add a `NotesList` component:

```
// pages/side-panel/src/components/NotesList.tsx
import { useState, useEffect, useCallback } from 'react';
import { notesStore, type Note } from '@extension/storage';

interface NotesListProps {
  onClose: () => void;  // Called when user clicks "Back"
}

export default function NotesList({ onClose }: NotesListProps) {
  const [notes, setNotes] = useState<Note[]>([]);
  const [loading, setLoading] = useState(true);

  // Load notes from storage:
  const load = useCallback(async () => {
    setLoading(true);
    const all = await notesStore.get();
```

```
    // Sort newest first:
    setNotes(all.sort((a, b) => b.createdAt - a.createdAt));
    setLoading(false);
  }, []);

  useEffect(() => {
    load();
  }, [load]);

  const handleClear = async () => {
    await notesStore.clearNotes();
    setNotes([]);
  };

  return (
    <div style={{ display: 'flex', flexDirection: 'column', height: '100%' }}>
      {/* Header bar: */}
      <div
        className="flex items-center justify-between px-3 py-2"
        style={{ borderBottom: '1px solid var(--line)' }}>
        <button type="button" onClick={onClose} className="label-mono"
          style={{ color: 'var(--muted)', background: 'none', border: 'none', cursor:
'pointer' }}>
          ← Back
        </button>
        <span className="label-mono" style={{ fontWeight: 700 }}>Notes</span>
        {notes.length > 0 && (
          <button type="button" onClick={handleClear} className="label-mono"
            style={{ color: 'var(--muted)', background: 'none',
                     border: '1px solid var(--line)', borderRadius: 4,
                     padding: '2px 8px', cursor: 'pointer' }}>
            Clear
          </button>
        )}
      </div>

      {/* Notes list: */}
      <div style={{ flex: 1, overflowY: 'auto', padding: 8 }}>
        {loading && (
          <div className="label-mono text-center py-4" style={{ color: 'var(--muted)' }}>
            Loading…
          </div>
        )}
        {!loading && notes.length === 0 && (
          <div style={{ textAlign: 'center', paddingTop: 40 }}>
            <div className="label-mono" style={{ color: 'var(--muted)' }}>No notes yet</div>
          </div>
        )}
        {!loading && notes.map(note => (
          <div key={note.id}
            style={{ background: 'var(--glass)', border: '1px solid var(--line)',
                     borderRadius: 6, padding: '8px 10px', marginBottom: 6 }}>
```

```
            <div style={{ fontSize: 12, color: 'var(--text)', marginBottom: 4 }}>
              {note.content}
            </div>
            <div className="label-mono" style={{ color: 'var(--muted)' }}>
              {new Date(note.createdAt).toLocaleString()}
            </div>
          </div>
        ))}
      </div>
    </div>
  );
}
```

Add it to `SidePanel.tsx` with a new `showNotes` state, following the same pattern as `WatchList` and `ScheduledTaskList`.

Part VIII — Reference

35. Full File Manifest

File	Purpose
`chrome-extension/manifest.js`	Generates manifest.json — extension identity and permissions
`chrome-extension/src/background/index.ts`	Service worker entry — handles all Chrome events and port connections
`chrome-extension/src/background/agent/executor.ts`	Orchestrates the three-agent loop
`chrome-extension/src/background/agent/planner.ts`	Strategy agent (LangChain, structured output)
`chrome-extension/src/background/agent/navigator.ts`	Execution agent (LangChain, tool use)
`chrome-extension/src/background/agent/validator.ts`	Quality-check agent
`chrome-extension/src/background/agent/types.ts`	AgentContext, ExecutionState, Actors enums
`chrome-extension/src/background/agent/actions/schemas.ts`	Zod schemas for every action
`chrome-extension/src/background/agent/actions/builder.ts`	Action implementations and registration
`chrome-extension/src/background/agent/prompts/templates/common.ts`	Shared security rules across all prompts
`chrome-extension/src/background/agent/prompts/templates/planner.ts`	Planner system prompt
`chrome-extension/src/background/agent/prompts/templates/navigator.ts`	Navigator system prompt

`chrome-extension/src/background/browser/context.ts`	Multi-tab management, CDP wrappers
`packages/storage/lib/base/base.ts`	createStorage factory
`packages/storage/lib/index.ts`	Exports all stores
`packages/storage/lib/profileStore.ts`	User personal data
`packages/storage/lib/resultsStore.ts`	Extraction results
`packages/storage/lib/uploadStore.ts`	Uploaded file contents
`packages/storage/lib/watchStore.ts`	Web monitor configurations
`packages/storage/lib/scheduledTaskStore.ts`	Scheduled task configurations
`pages/side-panel/src/SidePanel.tsx`	Root component, state machine, port connection
`pages/side-panel/src/SidePanel.css`	Theme variables, layout, glassmorphism
`pages/side-panel/src/index.css`	Global styles, Tailwind base
`pages/side-panel/src/components/ChatInput.tsx`	Input bar, send, mic, file attach
`pages/side-panel/src/components/FileUpload.tsx`	File picker, PDF→markdown conversion
`pages/side-panel/src/components/MessageList.tsx`	Chat history rendering
`pages/side-panel/src/components/ResultsCard.tsx`	Structured data display card
`pages/side-panel/src/components/ResultsList.tsx`	Results browser + diff mode
`pages/side-panel/src/components/WatchList.tsx`	Web watch management UI
`pages/side-panel/src/components/ScheduledTaskList.tsx`	Scheduled task management UI
`pages/options/src/Options.tsx`	Settings page root

---

36. LLM Provider Configuration

36.1 Opening Settings

Click the **Settings** icon (gear) in the side panel top-right, or from within the Dot extension, navigate to Options.

36.2 Supported Providers

Provider	API Key Source	Best Models
Anthropic	console.anthropic.com	claude-haiku-4-5 (nav), claude-sonnet-4-6 (planner)
OpenAI	platform.openai.com	gpt-4o-mini (nav), gpt-4o (planner)
Google Gemini	aistudio.google.com	gemini-2.5-flash (nav), gemini-2.5-pro (planner)
Ollama	localhost (no key needed)	qwen2.5:14b, mistral-small

Groq	console.groq.com	llama3-70b-8192

36.3 Model Assignment

In Settings, you assign:

- **Navigator model** — runs every step, should be fast and cheap
- **Planner model** — runs less frequently, benefits from stronger reasoning

A good cost-effective setup: Navigator = Gemini Flash (very cheap, fast), Planner = Claude Sonnet (strong reasoning, runs rarely).

---

37. Troubleshooting Guide

Extension won't load

```
Error: Could not load manifest
```

→ Run `pnpm build` first. The `dist/` folder must exist.

"No active tab found"

→ A Chrome system page is focused (`chrome://extensions/` , `chrome://settings/`). Switch to a real website and try again. The extension cannot run on internal Chrome pages.

Type-check fails with "Property X does not exist"

```
helper.ts(24,36): error TS2339: Property 'completionWithRetry' does not exist
```

→ This is a **pre-existing upstream error** in the LangChain types. It does not prevent the build from succeeding. Run `pnpm build` — it will complete despite the type-check failure.

Service worker goes idle mid-task

→ Long tasks (10+ minutes) may exceed Chrome's service worker timeout. The heartbeat mechanism mitigates this but cannot prevent all cases. For very long tasks, keep the side panel visible and active.

Alarm doesn't fire

→ Chrome alarms require the extension to be enabled. Check `chrome://extensions/` . Also, alarms don't fire when the browser is closed — open Chrome for scheduled alarms to trigger.

PDF upload shows no text

→ Some PDFs are scanned images, not text. `pdfjs-dist` can only extract text from PDFs that contain actual text layers, not from image-based scans.

---

38. Extending Dot

38.1 Adding a New Provider

1. Add the LangChain package for your provider: `pnpm add @langchain/new-provider -F chrome-extension`
2. Add a case to `createChatModel` in `helper.ts`
3. Add the provider to the options page settings form

38.2 Adding a New Workflow Template

```
// chrome-extension/src/background/workflows/my-workflow.ts
export const myWorkflow = {
  id: 'my-workflow',
  label: 'My Workflow',
  description: 'What this workflow does',
  params: [
    { key: 'target', label: 'Target URL', type: 'url' },
    { key: 'count', label: 'How many items', type: 'number', default: 10 },
  ],
  buildTask: (params: Record<string, string>) =>
    `Go to ${params.target} and extract ${params.count} items with all available details.`,
};
```

Then add it to `WorkflowPicker.tsx` and the workflow registry.

38.3 Adding a New Store

Follow the pattern in Part VII, Section 33. Key checklist:

- [ ] Define TypeScript interface
- [ ] Create storage with `createStorage`
- [ ] Add domain-specific methods
- [ ] Export from `packages/storage/lib/index.ts`
- [ ] Run `pnpm -F "@extension/storage" type-check`

38.4 Adding a New Action

Follow the pattern in Part VII, Section 32. Key checklist:

- [ ] Define schema in `schemas.ts`
- [ ] Export schema from `schemas.ts`
- [ ] Import and implement in `builder.ts`
- [ ] Add to the `actions` array
- [ ] Update the navigator system prompt to explain when/how to use it
- [ ] Run `pnpm -F chrome-extension type-check`

Closing Notes

This documentation covers every layer of Dot: from Chrome extension fundamentals to LLM agent architecture, from storage abstractions to React state management, from feature implementation to extending the system yourself.

The key insight unifying all of it: **a capable AI agent is not a monolithic system — it is a composition of small, well-defined pieces.** A schema defines a contract. An action implements it. A store persists state. A component

renders it. A prompt guides the LLM to use it well. Combine these patterns, and you can build almost any browser automation capability imaginable.

The entire codebase is open source at github.com/Kali2007thecodemaster/dot-browser — read it alongside this document, and every piece will fall into place.

Documentation written with Claude Sonnet 4.6.
Dot is built on Nanobrowser (Apache 2.0).
Apache License 2.0 — see LICENSE.